# vAzhure Racing Hub

## Firmware Upload Instructions

This guide walks you through the complete process of flashing firmware onto STM32 microcontrollers used in the vAzhure Racing Hub motion controller system. You will program one Master board and up to four Slave boards, then verify that everything works with the vAzhure Racing Hub software.

Before you begin, make sure you have the following items ready:

- An ST-Link V2 (or compatible) programmer/debugger dongle
- One STM32F103C8 (or compatible) board designated as the Master
- Up to four STM32F103C8 boards designated as Slaves (Front-Left, Rear-Left, Rear-Right, Front-Right)
- USB cables to connect the ST-Link to your PC
- Jumper wires for connecting the ST-Link to each STM32 board
- The assembled PCB where the programmed boards will be installed

***Warning:*** Always double-check the wiring connections before applying power. Connecting VCC or signal pins incorrectly may damage your STM32 board or the ST-Link programmer.

# 1. Install Required Software

You need two pieces of software installed on your computer before you can flash the firmware onto the STM32 boards. The Arduino IDE is used to compile and upload the firmware source code, while the ST-Link Utility provides the low-level USB driver that the Arduino IDE relies on for communication with the ST-Link programmer.

**Step 1.1:** Download and install the Arduino IDE

Go to the official Arduino IDE releases page and download the latest version for your operating system: https://github.com/arduino/arduino-ide/releases

Run the installer and follow the on-screen prompts. Once the installation is complete, launch the Arduino IDE to verify that it opens correctly.

**Step 1.2:** Download and install the ST-Link USB driver

Download the ST-Link Utility from STMicroelectronics: https://www.st.com/en/development-tools/stsw-link004.html

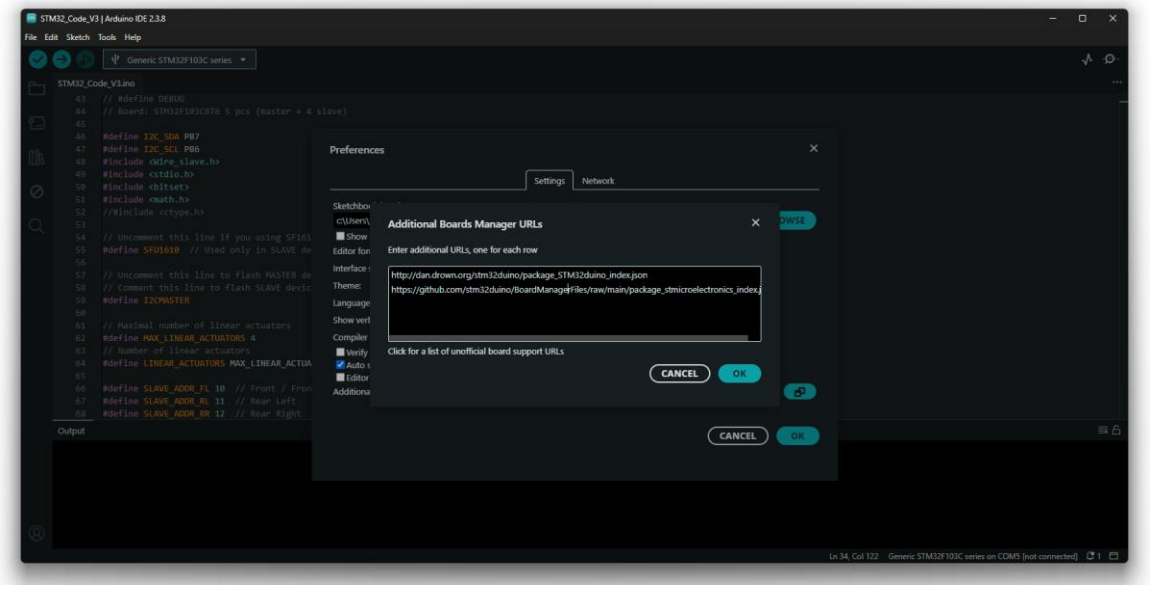
Install the software. This step is necessary even if you only use the Arduino IDE for uploading, because the ST-Link USB driver included in this package is required for the Arduino IDE to communicate with your ST-Link dongle.

# 2. Configure the Arduino IDE for STM32

By default, the Arduino IDE does not include support for STM32 microcontrollers. You need to add a third-party board index URL so that the IDE can download the necessary STM32 core files and board definitions.

**Step 2.1:** Add STM32 board manager URLs

Open the Arduino IDE and go to File > Preferences (on Windows/Linux) or Arduino IDE > Preferences (on macOS). In the Preferences window, find the field labeled "Additional boards manager URLs".



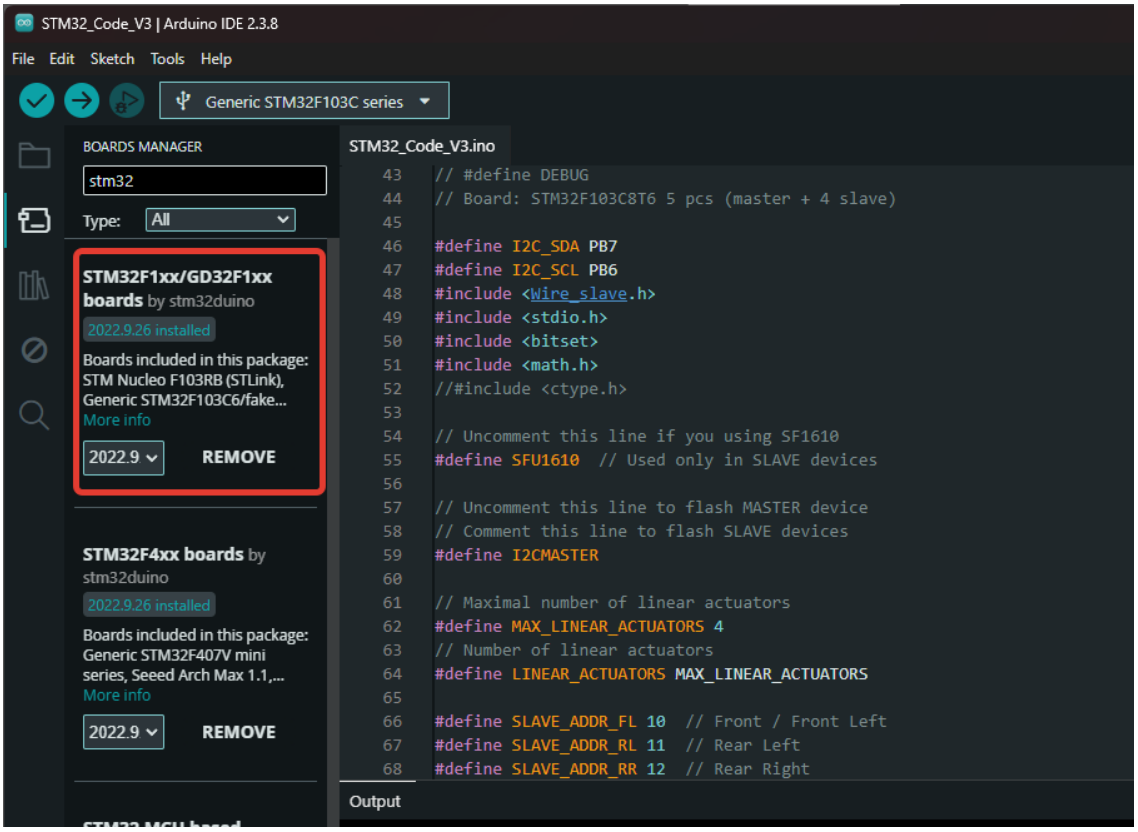

Append the following two URLs to that field. If there is already a URL present, separate them with a comma or place each URL on a new line:

```
http://dan.drown.org/stm32duino/package_STM32duino_index.json
https://github.com/stm32duino/BoardManagerFiles/raw/main/package_stmicroelectronics_index.json
```

Click OK to save the preferences.

### **Step 2.2:** Install the STM32 board package

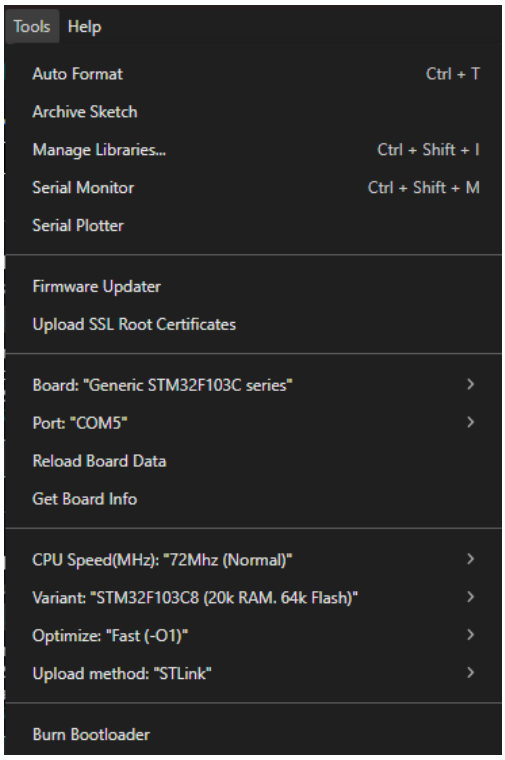
Go to Tools > Board > Boards Manager. In the search box at the top, type "STM32" and look for an entry named "STM32F1xx/GD32F1xx boards" in the list of results. Click the Install button next to it and wait for the download and installation to complete. This may take a few minutes depending on your internet speed.



**Step 2.3:** Set the board configuration in the Tools menu

Once the STM32 board package is installed, open the Tools menu and configure the following settings exactly as shown below. These settings must match for every firmware build in this guide:

| Setting | Value |
|---|---|
| Board | "Fake STM32F103C8" or "Generic STM32F103C series" |
| CPU Speed (MHz) | 72 MHz (Normal) |
| Optimize | Fast (-O1) |
| Upload Method | STLink |



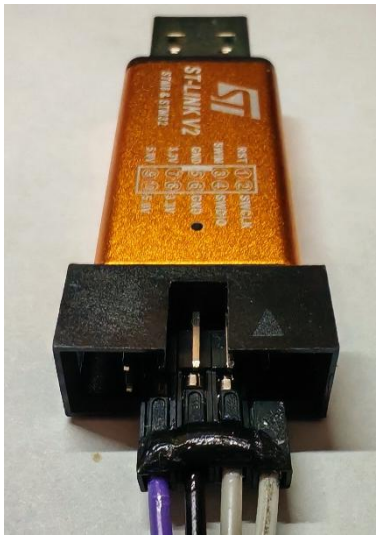
# 3. Wire the ST-Link to the STM32 Board

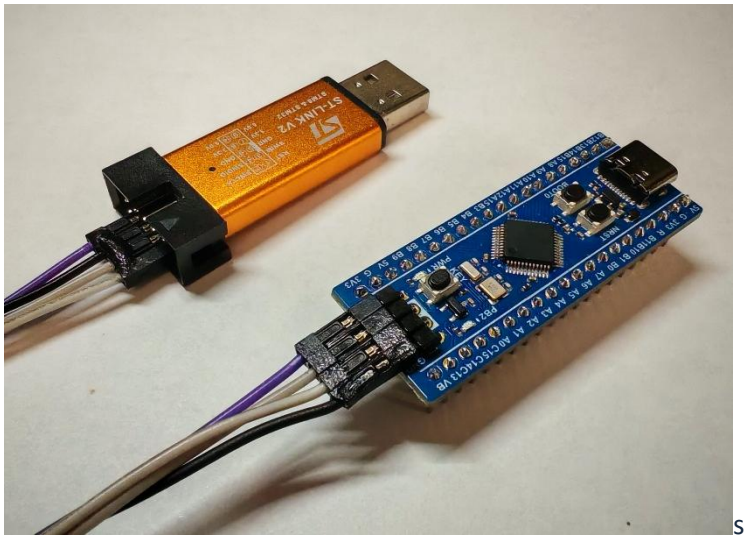
Before uploading any firmware, you need to physically connect your ST-Link programmer to the target STM32 board using jumper wires. The ST-Link communicates with the STM32 over a two-wire debug interface called SWD (Serial Wire Debug). In addition to the two data lines, you must also connect power and ground.

Use four jumper wires to make the following connections between your ST-Link dongle and the STM32 board:

| ST-LINK Pin | Signal | | STM32 Pin |
|---|---|---|---|
| | SWCLK | → | SCK |
| | SWDIO | → | DIO |
| | 3.3V | → | 3.3V |
| | GND | → | GND |

The following photos show how the wiring should look in practice:




s

***Note:*** Make sure the ST-Link is not connected to USB power while you are wiring the connections. Connect the USB cable only after all four wires are securely in place.

# 4. Upload the Master Firmware

The Master board is the central controller that communicates with all Slave boards over the I2C bus. It must be programmed first, because the Slave boards depend on the Master for coordination. The firmware source code is hosted on GitHub in a single sketch file.

**Step 4.1:** Download and open the firmware sketch

Open the following link in your browser and download the sketch file: STM32_Code_V3.ino

Open the downloaded .ino file in the Arduino IDE. The sketch will appear in the editor window.

**Step 4.2:** Enable the Master configuration

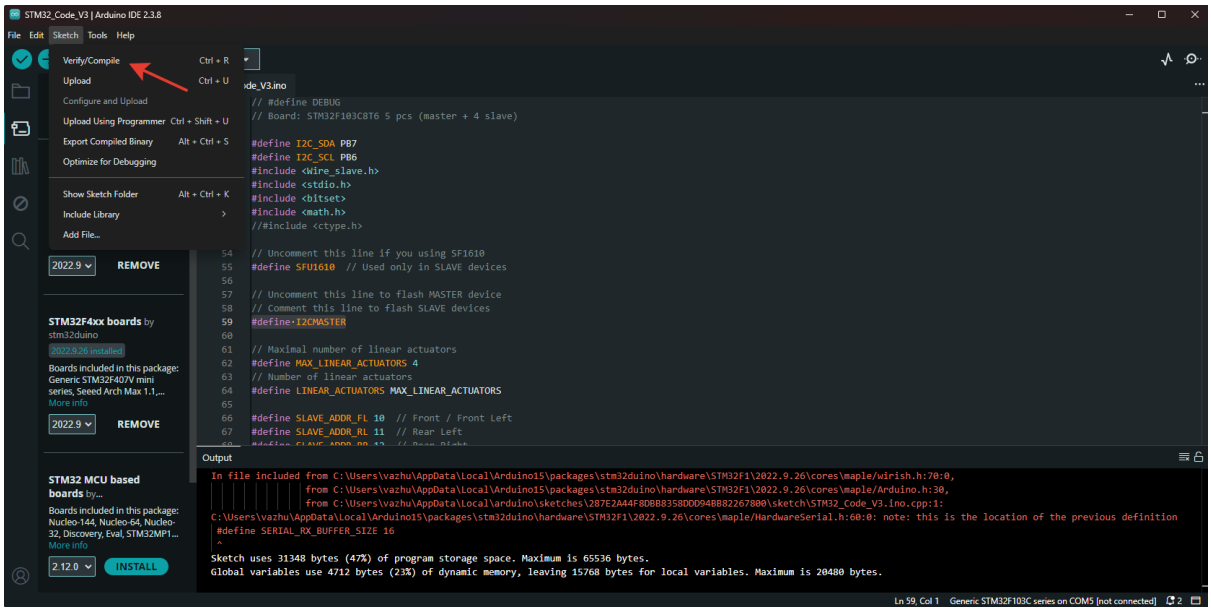
Near the top of the sketch file, locate the line that reads:

```
// #define I2CMASTER
```

Remove the two forward slashes at the beginning of the line so that it becomes:

```
#define I2CMASTER
```

This activates the Master mode in the firmware.



**Step 4.3:** Compile the Master firmware

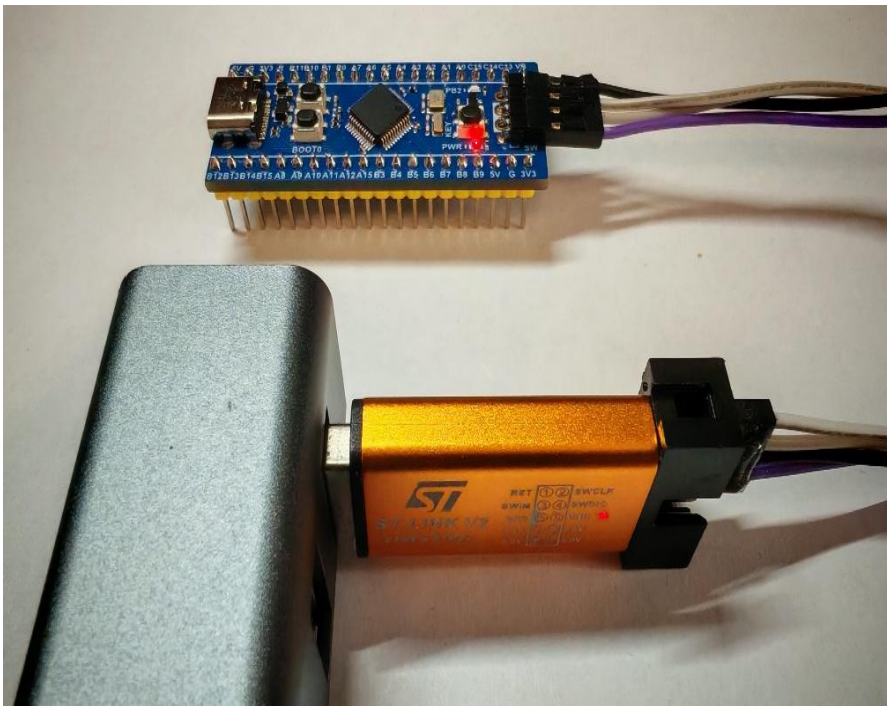
Click the Verify button (the checkmark icon) in the Arduino IDE toolbar, or press Ctrl+R. Wait for the compilation to finish. If everything is configured correctly, you will see a success message in the Output panel that looks like this:

```
Sketch uses 31348 bytes (47%) of program storage space. Maximum is 65536 bytes.
Global variables use 4712 bytes (23%) of dynamic memory, leaving 15768 bytes for
local variables. Maximum is 20480 bytes.
```

***Note:*** If the compilation fails with errors, double-check that the correct board (Generic STM32F103C series) is selected in the Tools menu and that the STM32 board package was installed successfully in Step 2.2.

**Step 4.4:** Upload the Master firmware to the board

Connect the ST-Link dongle to your PC using a USB cable. The ST-Link driver you installed in Step 1.2 should be recognized automatically by the operating system. Then, click the Upload button (the right-arrow icon) in the Arduino IDE toolbar, or press Ctrl+U.



The Arduino IDE will compile the sketch again (if any changes were made) and then flash it onto the STM32 board through the ST-Link. Wait for the upload process to complete. A successful upload will display a message in the Output panel confirming that the firmware was written to the flash memory.

```
Output
  0%◆◆◆◆◆◆◆◆◆◆◆◆◆◆ 24%◆◆◆◆◆◆◆◆◆◆◆◆◆◆ 48%◆◆◆◆◆◆◆◆◆◆◆◆◆◆ 73%◆◆◆◆◆◆◆◆◆◆◆◆◆◆ 97%◆◆ 100%
Memory programmed in 2s and 406ms.
Programming Complete.

MCU Reset.

Application started.
```

Once the upload finishes successfully, disconnect the ST-Link from the STM32 board. The Master board is now programmed and ready.

# 5. Upload the Slave Firmware

Each Slave board needs its own copy of the firmware compiled with a unique I2C address. The address is determined by which #define line is uncommented in the sketch. You will repeat the compile-and-upload cycle for each Slave board, changing only the address line each time.

**Step 5.1:** Switch from Master to Slave mode

In the Arduino IDE, go back to the top of the sketch file and comment out the I2CMASTER line by adding two forward slashes at the beginning:

```
// #define I2CMASTER
```

This disables Master mode and prepares the sketch for Slave mode configuration.

```
// Uncomment this line to flash MASTER device
// Comment this line to flash SLAVE devices
//#define I2CMASTER
```

**Step 5.2:** Program the Front-Left Slave (FL)

Uncomment the line for the Front-Left slave address by removing the forward slashes:

```
#define SLAVE_ADDR SLAVE_ADDR_FL
```

Make sure all other SLAVE_ADDR lines remain commented out. Only one address line should be active at a time.

```
#ifndef I2CMASTER
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// Uncomment a single line with desired Address to flash SLAVE device
#define SLAVE_ADDR SLAVE_ADDR_FL
//#define SLAVE_ADDR SLAVE_ADDR_RL
//#define SLAVE_ADDR SLAVE_ADDR_RR
//#define SLAVE_ADDR SLAVE_ADDR_FR
#endif
```

Connect the ST-Link to the Front-Left STM32 board using the same wiring diagram from Section 3. Click Verify to compile the sketch. You should see a success message similar to:

```
Sketch uses 34500 bytes (52%) of program storage space. Maximum is 65536 bytes.
Global variables use 4672 bytes (22%) of dynamic memory, leaving 15808 bytes for
local variables. Maximum is 20480 bytes.
```

```
Output
  0%◆◆◆◆◆◆◆◆◆◆◆◆ 22%◆◆◆◆◆◆◆◆◆◆◆◆ 44%◆◆◆◆◆◆◆◆◆◆◆◆ 66%◆◆◆◆◆◆◆◆◆◆◆◆ 89%◆◆◆◆◆◆ 100%
  Memory programmed in 2s and 578ms.
  Programming Complete.

  MCU Reset.

  Application started.
```

Click Upload to flash the firmware onto the Front-Left board. Wait for the upload to complete successfully.

**Step 5.3:** Program the remaining Slave boards

Repeat the process from Step 5.2 for each of the remaining Slave boards. For each board, comment out the previous SLAVE_ADDR line and uncomment the one corresponding to the current board. The complete list of Slave addresses is shown in the table below:

| Slave Device | Macro to Uncomment |
|---|---|
| Front-Left (FL) | #define SLAVE_ADDR SLAVE_ADDR_FL |
| Rear-Left (RL) | #define SLAVE_ADDR SLAVE_ADDR_RL |
| Rear-Right (RR) | #define SLAVE_ADDR SLAVE_ADDR_RR |
| Front-Right (FR) | #define SLAVE_ADDR SLAVE_ADDR_FR |

For example, to program the Rear-Left board, change the active line to:

```
#define SLAVE_ADDR SLAVE_ADDR_RL
```

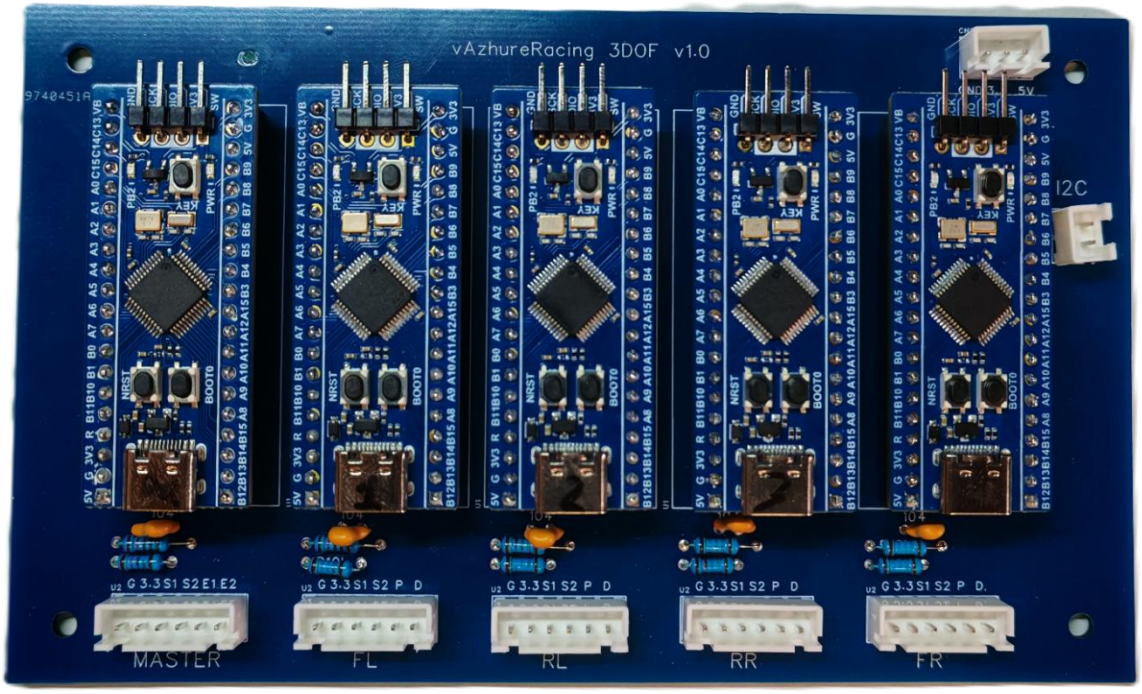
```
97  #ifndef I2CMASTER
98  ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
99  // Uncomment a single line with desired Address to flash SLAVE device
100 //#define SLAVE_ADDR SLAVE_ADDR_FL
101 #define SLAVE_ADDR SLAVE_ADDR_RL
102 //#define SLAVE_ADDR SLAVE_ADDR_RR
103 //#define SLAVE_ADDR SLAVE_ADDR_FR
104 #endif
```

Follow the same verify-and-upload cycle for each Slave: Rear-Left (RL), Rear-Right (RR), and Front-Right (FR). Connect the ST-Link to each board one at a time and upload the corresponding firmware.

> ***Note:*** It is easy to forget to change the address line between uploads. Always verify that only the correct SLAVE_ADDR line is uncommented before clicking Upload. Uploading the wrong address to a board will cause I2C communication conflicts later.

# 6. Install the Programmed Boards on the PCB

Once all boards have been successfully programmed, carefully install them onto the motion controller PCB. Pay close attention to the physical orientation and the slot assignments for each board. The Master board must be placed in its designated position, and each Slave board must occupy the correct slot matching the address that was programmed into it.




Verify that the boards are seated properly in their sockets and that no pins are bent or misaligned. A loose connection can cause intermittent failures that are difficult to diagnose later.

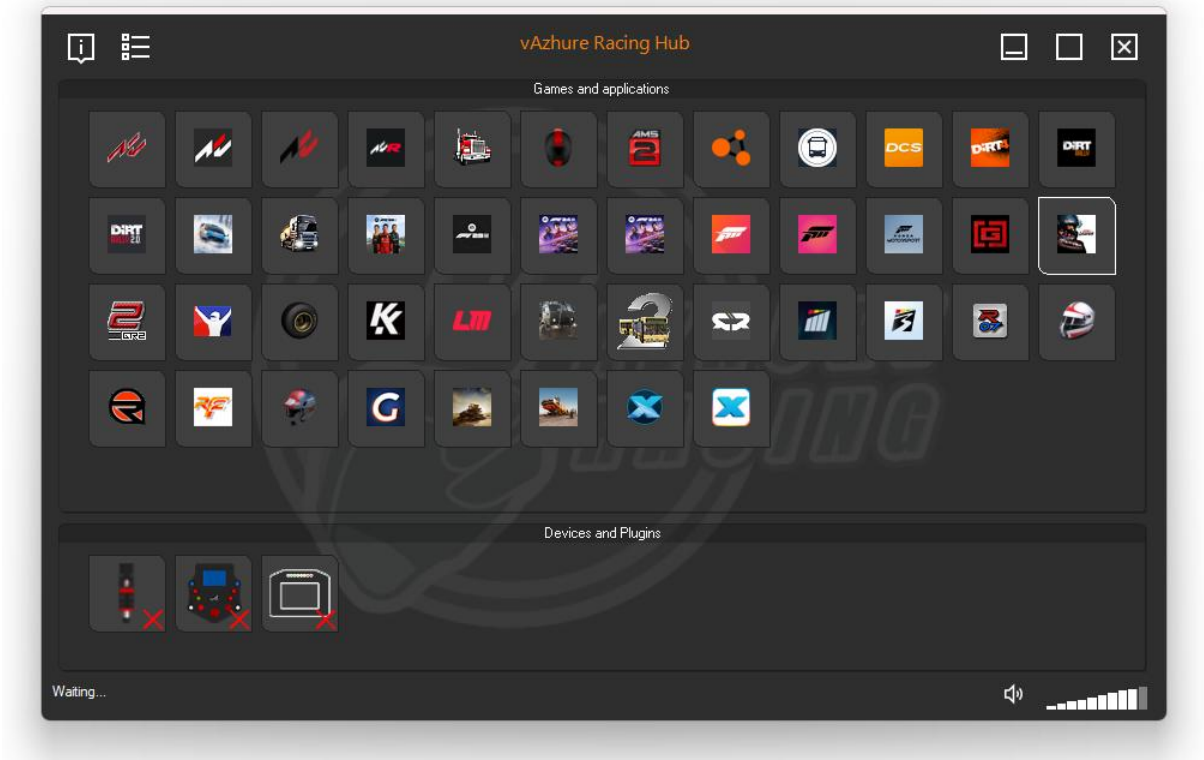
# 7. Install and Run the vAzhure Racing Hub Software

With the hardware fully assembled, the final step is to install the vAzhure Racing Hub application on your PC. This software communicates with the Master STM32 board over USB and provides the user interface for configuring and controlling the motion platform.

**Step 7.1:** Download and install the software

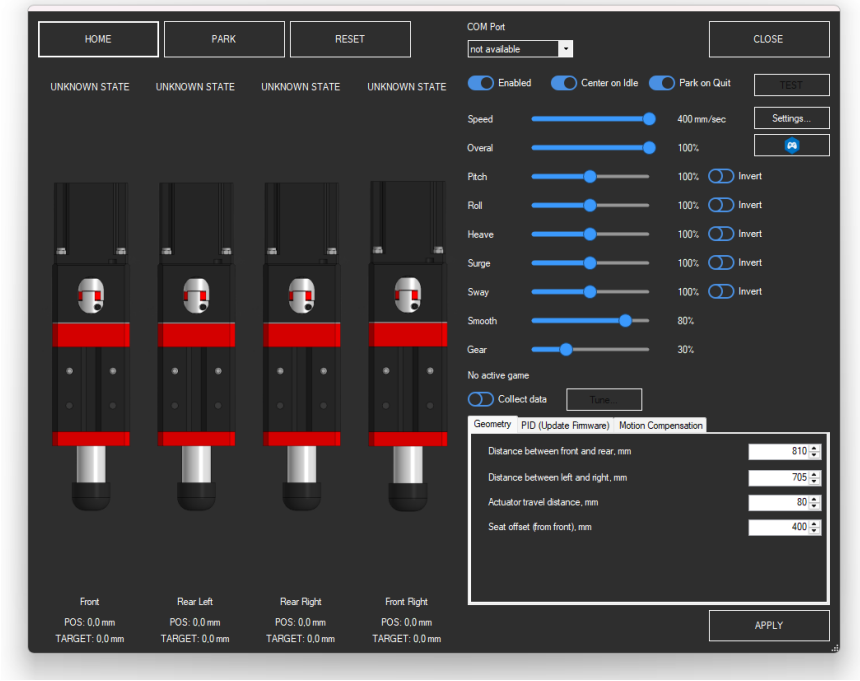
Go to the releases page on GitHub and download the latest version of the vAzhure Racing Hub: https://github.com/vazhure/vAzhureRacingHub/releases

Extract the archive and run the installer (or launch the executable directly if it is a portable version). Follow the installation prompts to complete the setup.




**Step 7.2:** Connect the Master board to your PC

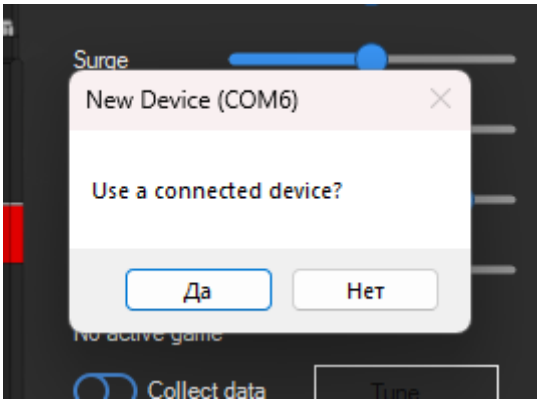
Use a USB cable to connect the Master STM32 board on the PCB to your PC. The Master board should be recognized as a new COM port (on Windows).



### Step 7.3: Detect the device in the software

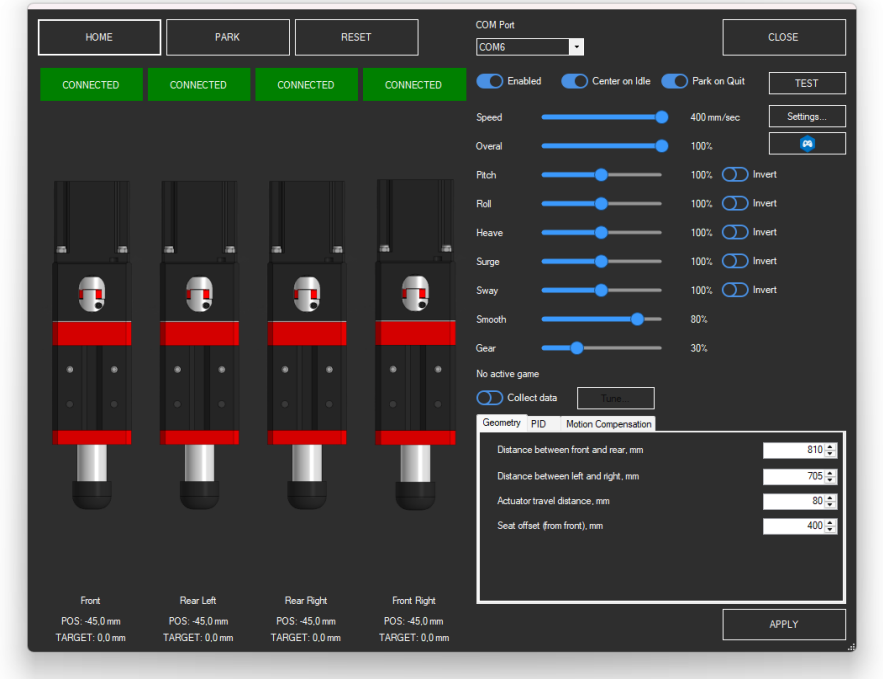
Launch the vAzhure Racing Hub application. Click the device detection icon in the toolbar to scan for connected hardware.

When the software detects the Master board, a dialog will appear asking whether you want to register the new device. Click "Yes" to confirm.




### Step 7.4: Verify the setup

If the PCB is assembled correctly and all firmware was flashed properly, the software should display the full system status with all Slave boards reporting in. You should see indicators for each axis (Front-Left, Front-Right, Rear-Left, Rear-Right) showing that they are online and responding to commands from the Master.



If any boards are not showing up, go back and verify the following:

- The correct firmware was uploaded to each board (check the SLAVE_ADDR lines)
- All boards are properly seated in their PCB sockets
- The USB cable and connection are working (try a different cable or port)
- The ST-Link USB driver is installed (required for COM port detection)

Congratulations! Your vAzhure Racing Hub motion controller is now fully set up and ready to use.